

2.ARITHMETIC AND LOGICAL OPERATIONS ON IMAGES

AIM:

To study and implement the arithmetic and logical operations on images using MATLAB.

INTRODUCTION:

Image processing is the method of performing operations on images to enhance their quality or extract meaningful information. It allows computers to analyze visual data, much like how humans see and interpret the world. This field is essential in applications like medical scans, satellite imagery, facial recognition, and robotics. The typical image processing workflow includes steps like acquiring the image, enhancing it (by reducing noise or adjusting brightness), segmenting important regions, extracting features like edges or shapes, and finally interpreting or classifying the content.

BASIC IMAGE OPERATIONS:

Basic image operations are foundational techniques in digital image processing, allowing for manipulation, analysis, and enhancement of image data. Some of the most common operations include:

Image Reading and Display:

Images are loaded into memory using functions like `imread()` and displayed using `imshow()` in MATLAB.

Example: `img = imread('image.jpg');`
`imshow(img);`

Color to Grayscale Conversion:

Converts an RGB image to a grayscale image by averaging or weighted summation of the color channels.

Example : `grayImg = rgb2gray(img);`

Image Resizing:

Changes the dimensions of an image using interpolation techniques.

Example: `resizedImg = imresize(img, [256 256]);`

Image Cropping:

Extracts a specified portion of the image.

Example: `croppedImg = img(50:150, 100:200, :);`

Image Rotation and Flipping:

Rotates or flips the image to a desired orientation.

Example: `rotatedImg = imrotate(img, 90);`

`flippedImg = flip(img, 2); % Horizontal flip`

Image Arithmetic:

Involves pixel-wise operations such as addition, subtraction, multiplication, or division between images or between an image and a scalar.

Example: `brighterImg = img + 50; % Increases brightness`

Thresholding (Binary Conversion):

Converts an image to a binary format based on a threshold value.

Example: `bwImg = imbinarize(grayImg);`

Image Filtering:

Applies filters (e.g., blurring, sharpening, edge detection) to enhance or extract features.

Example: `blurredImg = imgaussfilt(grayImg, 2);`

THEORY:

1. Arithmetic Operations:

- **Addition:** Enhances image brightness or blends images.

$$G(x,y)=F_1(x,y)+F_2(x,y) \quad G(x,y)=F_1(x,y)+F_2(x,y)$$

If the sum exceeds 255 (for 8-bit), it is clipped to 255.

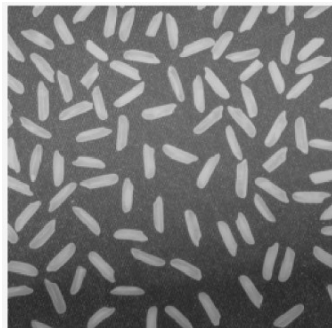
Code:

```
img1 = imread('cameraman.tif');  
img2 = imread('rice.png');  
a = img1 * 0.3 + img2 * 0.7;  
imshow(a);  
imshow(img1)  
imshow(img2)
```

Input image: img1



Input image: img2



output image: a



- **Subtraction:** Used for motion detection or DSA (Digital Subtraction Angiography).

$$G(x,y)=F_1(x,y)-F_2(x,y)$$

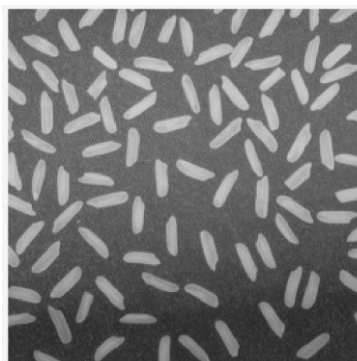
Code:

```
img1 = imread('cameraman.tif');
img2 = imread('rice.png');
a = img1 * 0.7 - img2 * 0.3;
imshow(a);
imshow(img1)
imshow(img2)
```

Input image: img1



Input image: img2



Output image: a



- **Multiplication/Division with Constant:**
 - $G = \alpha \cdot F$: Brightens or darkens image.
 - $G = F / \alpha$: Reduces brightness.

Code:

```
img = imread('cameraman.tif');
img = double(img);
alpha = 1.5;
bright_img = alpha * img;
dark_img = img / alpha;
bright_img = uint8(bright_img);
dark_img = uint8(dark_img);
imshow(uint8(img)), title('Original');
imshow(bright_img), title('Brightened');
imshow(dark_img), title('Darkened');
```

Output :

Original:



Brightened:



Darkened:



- **Image Averaging:** Reduces random noise by averaging several noisy images of the same scene:

$$G(x,y) = \frac{1}{N} \sum_{i=1}^N F_i(x,y)$$

2. Logical Operations:

- **AND:**
 - Masking
 - Region of Interest (ROI) extraction
 - Intersection of two binary images

- **OR:**
 - Merging features
 - Enlarging selected ROI
- **XOR:**
 - Change detection
 - Comparing image similarity
- **NOT:**
 - Produces image negative
 - Enhances contrast for some features

Logical operations are typically applied to **binary or grayscale images** in a **bitwise** manner.

APPLICATIONS:

- **Medical Imaging:** DSA for artery block detection.
- **Computer Vision:** Object tracking using subtraction.
- **Robotics:** Masking important regions using logical operations.
- **Remote Sensing:** Image fusion using addition/multiplication.
- **Noise Reduction:** Image averaging for cleaner visuals.

SOFTWARE USED:

- MATLAB R202X and Image Processing Toolbox

RESULTS:

- Successfully implemented image **addition, subtraction, multiplication, division**.
- Applied **AND, OR, XOR, NOT** operations on grayscale and binary images.
- Demonstrated image **masking, blending**, and **noise reduction** via averaging.

(Screenshots of output images can be pasted here)

CONCLUSION:

Arithmetic and logical operations are essential tools in image processing. They help enhance, extract, or compare specific image features. These operations are efficient, fast, and widely applicable across medical, industrial, and scientific domains.